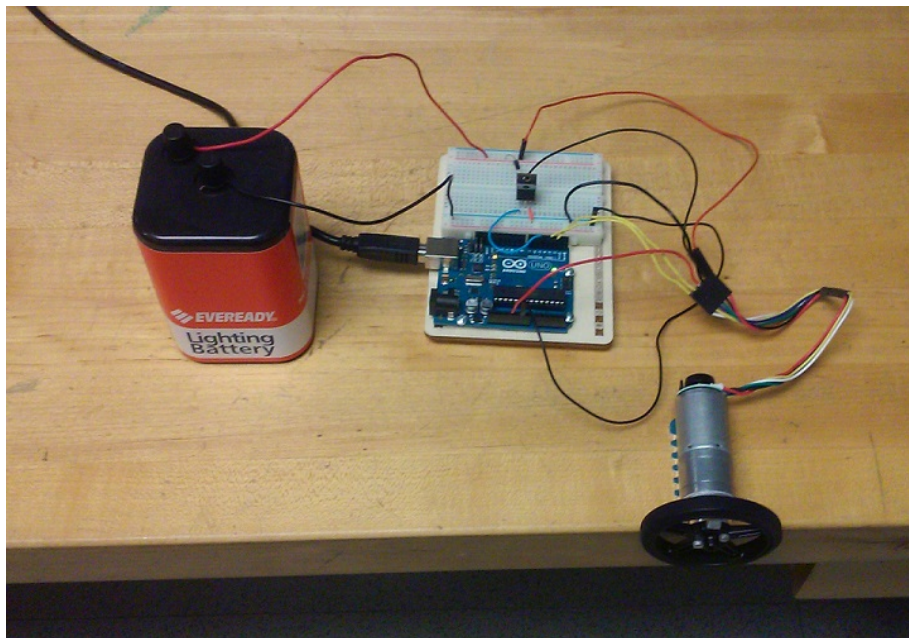


# Speed Control of a DC motor

## 1 Introduction

In this classwork we will model a simple DC motor for an input of armature voltage and an output of rotational speed. The motor's angular position (and in turn its speed) is determined by a quadrature encoder. The encoder pulses are counted on the Arduino board via two of the board's Digital Inputs (each digital channel can be either an input or an output). The Arduino board is also used for controlling the speed of the motor. Specifically, one of the board's Digital Outputs is employed to switch the transistors of an H-bridge IC circuit on and off, thereby connecting and disconnecting the motor to a DC Voltage source. The Arduino board will also communicate the recorded data to the serial monitor. The logic for estimating the motor's speed based on encoder counts is implemented within the Arduino IDE. In the second part, the logic for controlling the motor's speed will also be implemented in the Arduino IDE.



## 2 Time-Response Analysis of a DC Motor

The purpose of this part is to build intuition regarding the operation of an armature-controlled DC motor. This part also generates a blackbox model for the motor based on its step response. This type of model is compared to a physics-based model, the details of which can be found in the class lectures (ED2).

## 2.1 Equipment needed

- Arduino board (e.g. Uno, Mega 2560, etc.)
- Breadboard
- DC motor with quadrature encoder
- Power supply (lantern battery for example)
- Dual H-bridge motor driver IC (L293D or SN754410 for example)
- Jumper wires

## 2.2 Modeling from first principles

In order to generate a physics-based model of the motor, we need to consider a simplified version of its workings. Figure (1) represents an electric equivalent circuit of the armature and the free-body diagram of the rotor. For this example, we will treat the voltage source ( $V$ ) applied

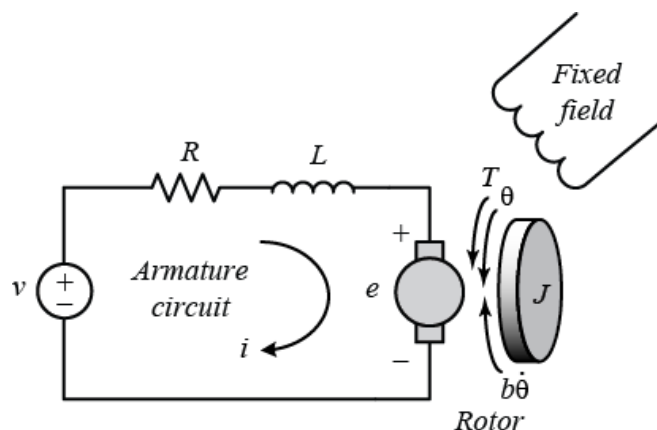


FIGURE 1 – Physics-based model of a DC motor

to the motor's armature as the input, and the rotational speed of the shaft  $d\theta/dt$  as the output. For modelling purposes, the rotor and shaft are assumed to be rigid. We further assume a viscous friction model, that is, that the friction torque is proportional to shaft angular velocity.

## 2.3 Time response experiment

In this experiment we will generate a model for an armature-controlled DC motor based on its step response. Therefore, we will generate a model for the motor based on its observed response, without considering the underlying physics of the motor. This is sometimes referred to as a blackbox model or a data-driven model. After we have generated such a model, we will attempt to explain what we have observed based on our understanding of the underlying physics. In this process we will learn how to interface with a DC motor equipped with a quadrature encoder.

### 2.3.1 Hardware setup

In this experiment we will control our motor through one the board's digital outputs. Since the board cannot supply enough current (only about 40 milliamps) to drive most motors directly, we will use the low-power signal from the board to connect and disconnect the motor to a higher-power source, (i.e. a battery/generator). Specifically, the digital output will be used to switch the transistors of the motor driver on and off, and will behave like a closed or open switch thereby completing the circuit and causing the motor to spin or to stop. Since the motor is an inductive load, when we attempt to switch the motor off and it continues to spin (due to its inertia) the motor will generate a back emf (a voltage). This back emf can damage our transistor. In order to prevent this back emf from causing damage, the dual H-bridge motor driver includes a "flyback" diode in parallel with each transistor. The diode will only allow current to flow in one direction, thereby protecting the rest of the circuit.

**Question 1** Why do we need to use external battery to run the motor ?

#### H-bridge : Dual-Direction Control of the motor

In order to drive each motor in two directions, we need to build an H-bridge circuit as shown in figure (8) The H-bridge circuit works as follows :

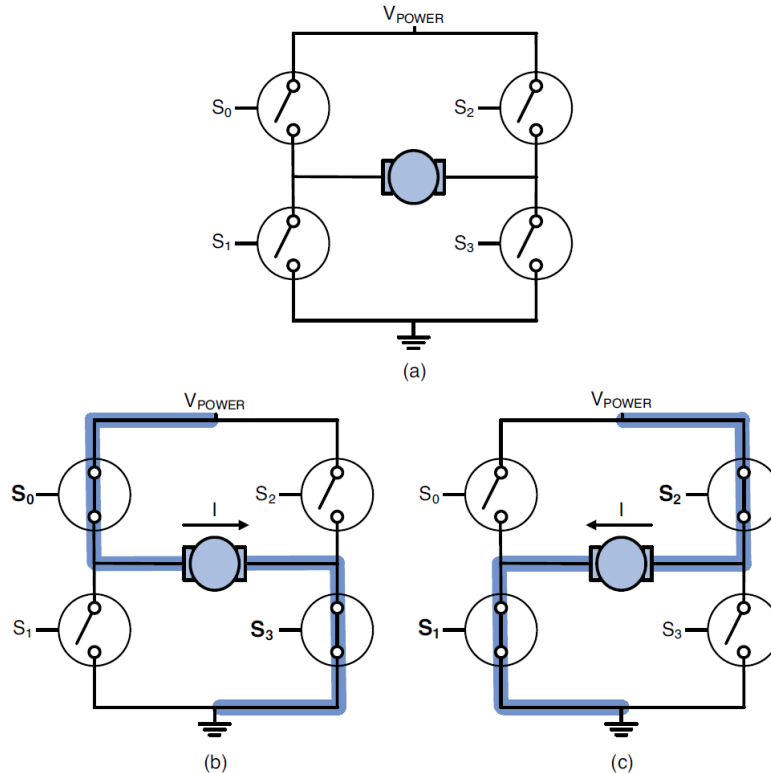


FIGURE 2 – Controlling motor direction with an H bridge

- S0 and S3 closed, S2 and S1 open : Current flows through the motor from left to right. This is depicted in Figure (8-b).
- S2 and S1 closed, S0 and S3 open : Current flows through the motor from right to left. This is depicted in Figure (8-c).
- S0 and S2 open : The motor's position is held in place.

It's common to encounter H bridges constructed out of MOSFETs, this is shown in figure (3). Rather than construct an H Bridge from discrete transistors, it's simpler to use an integrated

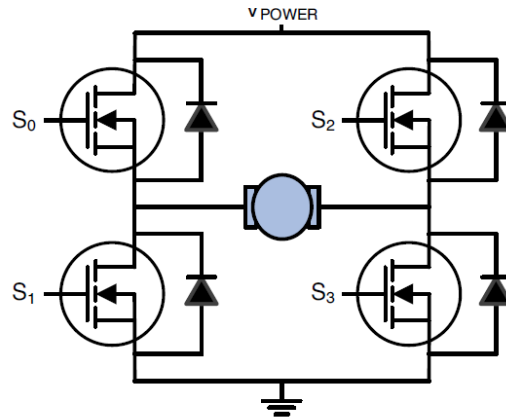


FIGURE 3 – H bridge consisting of MOSFETs and flyback diode in parallel

circuit such as the L293D or SN754410.

### L293D Motor Driver

L293D is a typical Motor driver or Motor Driver IC which allows DC motor to drive on either direction. L293D is a 16-pin IC which can control a set of two DC motors simultaneously in any direction. It means that you can control two DC motor with a single L293D IC. In a single L293D chip there are two h-Bridge circuit inside the IC which can rotate two dc motor independently . The L293D is designed to provide bidirectional drive currents of up to 600 mA (per channel) at voltages from 4.5 V to 36 V.

The schematic of the L293D Motor Driver is shown in figure (4).

Enable-1 and Enable-2 should be connected to 5 volts, they allow us to use the left and right motor, respectively. Input1, input2, input3 and input4 should be connected to a pwm Arduino pin, by varying the value of PWM we can control the speed and the direction of the motor. for example if input1 is set to high and input2 is set to low, the left motor will turn on in one direction, and if we swap the values the left motor will turn in the opposite direction. Same thing applies for input3 and 4 to control the right motor. **The L293 can get too hot and risk melting the breadboard, therefore it is recommended to use a heat sink.**

We are able to achieve approximately continuous control of the motor's speed using only a DC source by employing Pulse-Width Modulation (PWM). Recall, a PWM approach alternately turns the motor "on" and "off". Since the motor has dynamics (inertia, friction, etc.), it doesn't

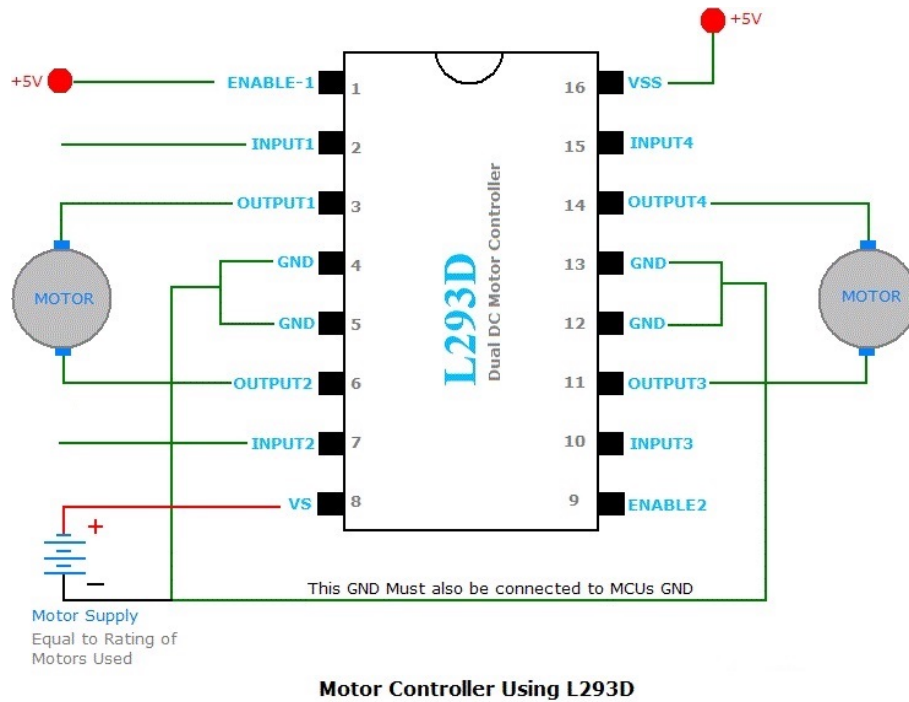


FIGURE 4 – L293D Motor Driver

instantaneously reach top speed when turned on and doesn't immediately come to a stop when turned off. It takes time for the motor to react. This can be used to our advantage if the PWM frequency is sufficiently fast. It's as if the motor is filtering the PWM command. In this way, the speed of the motor can be controlled continuously by varying the percent of time the PWM signal is on compared to the overall period (the duty cycle).

We will also employ the Arduino board for sensing the position/speed of the motor. There are different means for achieving this with one of the most popular being the optical encoder (see figure 5). An optical encoder consists of an emitter (a light source, like an LED) and a detector (a phototransistor). Within an encoder there is a piece of material that alternately blocks and passes light as the material is passed between the emitter/detector pair. When the phototransistor detects the light its output goes high, and when it doesn't detect light its output goes low. By counting these pulses, the sensor indicates the displacement of the material between the emitter and detector. Furthermore, by timing the frequency of these pulses, the sensor can be used to indicate speed. By employing two emitter/detector pairs that are slightly offset, the optical encoder can also be used to determine the direction of motion. In the following rotary example shown in figure (6), when the disk is rotating counterclockwise the disk will encounter Pair A first and Pair B second, thereby causing channel A to go low before channel B goes low. When the disk is rotating clockwise, then channel B will go low first. This type of configuration is typical of what is referred to as quadrature. Another commonly employed transducer is a Hall-effect sensor (figure 7). A Hall-effect sensor is able to detect a magnetic field. By detecting the passing of gear teeth or a magnet, the Hall-effect sensor can generate a pulse-train output similar to that produced by an optical encoder. Furthermore, employing offset Hall-effect sensors

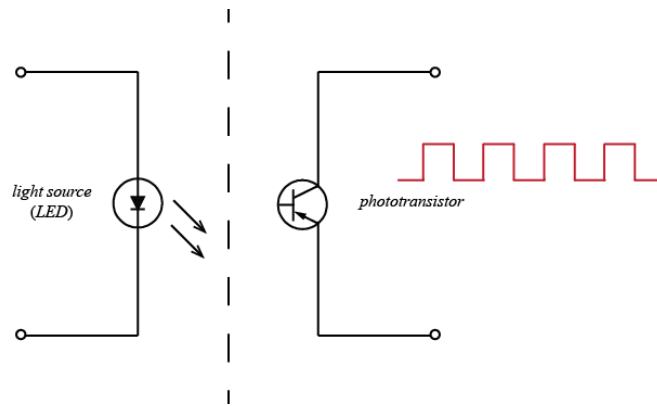


FIGURE 5 – Optical encoder scheme

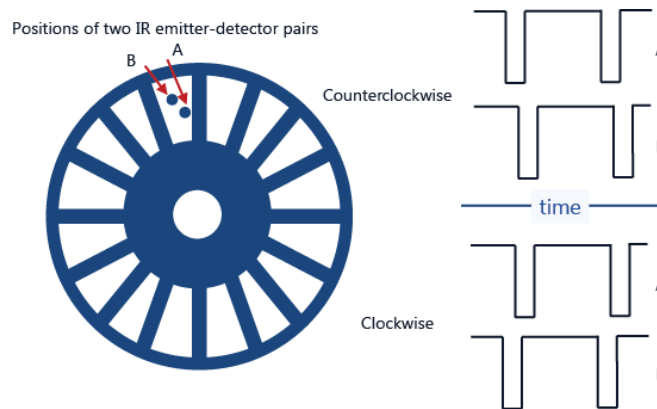


FIGURE 6 – Encoder's orientation detection using two channels A and B

can generate a quadrature output for determining the direction of motion. In this activity we will

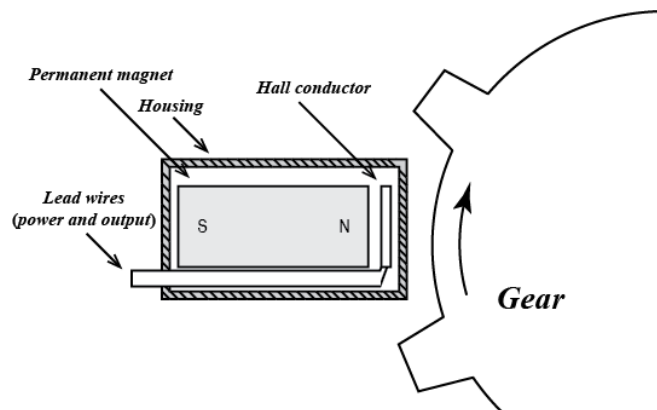


FIGURE 7 – Hall effect encoder

use a motor with a hall-effect encoder that can be driven by a 12-V power supply. The encoder provides around 1200 counts per revolution of the output shaft of the gearbox (if you count both rising and falling edges).

**Question 2** What are the two types of encoder that are commonly used ? What is the difference between them ?

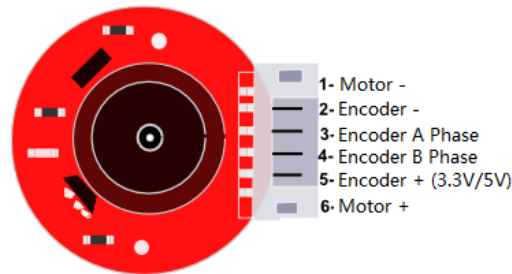


FIGURE 8 – Motor with hall effect encoder wires function

### 2.3.2 Software setup

In this experiment, we will employ The Arduino IDE to control the motor through the H-bridge motor driver, to read the encoder output, and to plot the data in Matlab. Before writing any code, make sure that you built the circuit following the connections shown by the professor. Now run the motor at full speed we will be able to estimate the transfer function of the motor (plant).

The encoder will measure count the number of pulses, therefore we get access to the angular displacement and the velocity.

In order to measure the data, we need to include interrupts in our code. The reason is that an interrupt makes sure that the processor responds quickly to important events. When a certain signal is detected, an Interrupt (as the name suggests) interrupts whatever the processor is doing, and executes some code (a special function) designed to react to whatever external stimulus is being fed to the Arduino (in our case we will count the number of pulses detected by the encoder). Once that code (special function) has wrapped up, the processor goes back to whatever it was originally doing as if nothing happened. We have to make sure that the special function should be as short as possible, for not causing the processor to wait for too long and therefore miss some pulses from the encoder. If we don't include an interrupt, since the motor run at a fast rate, we will miss some pulses from the encoder because the Arduino will be busy doing other things, and therefore our data will not be accurate.

The Arduino code for a simple interrupt is shown in figure (9), the function used to set an interrupt is `attachInterrupt(pin,function,CHANGE)`. The function `attachInterrupt` takes 3 arguments, first argument "pin" is the internal interrupt pin, it can take only two value 0 or 1 : 0 if you use pin 2, 1 if you use pin 3. Second argument "function" is the name of the function that is called when an interrupt event occurs. Third argument "Change" is the type of interrupt, this means that an interrupt occurs if arduino detects any change (0 to 1 or 1 to 0) in the interrupt



```

SimpleInterrupts
// Sample code using interrupts
unsigned long count=0;
unsigned long timep,time,etime;

void setup()
{
  Serial.begin(9600);
  //connect channel A to pin 3
  pinMode(3,INPUT);
  attachInterrupt(1,transition,CHANGE); //interrupt(0,*,*) for pin 2; interrupt(1,*,*) for pin 3
  //set the initial time
  timep=micros();
}

void transition()
{
  count++;
}

void loop()
{
  time = micros();
  etime=time - timep;
  if (etime>1000000) //1 second
  {
    Serial.println(count); // each second we will print the number of counts
    timep = time; //reset timer
  }
}

```

FIGURE 9 – Simple interrupt code

pin. The example code shown in figure (9) print the values of the motor count at each second (of course you need to run the motor first so the encoder will start counting). The code can be downloaded from [savoir->Automatique->practical work->PW2->simpleInterupt.ino](#)

**Question 3** Write the arduino code to run the motor, add an interrupt code to get the values and convert them into RPM (unit)

**Question 4** Plot the velocity (RPM) vs time (sec) and estimate the transfer function

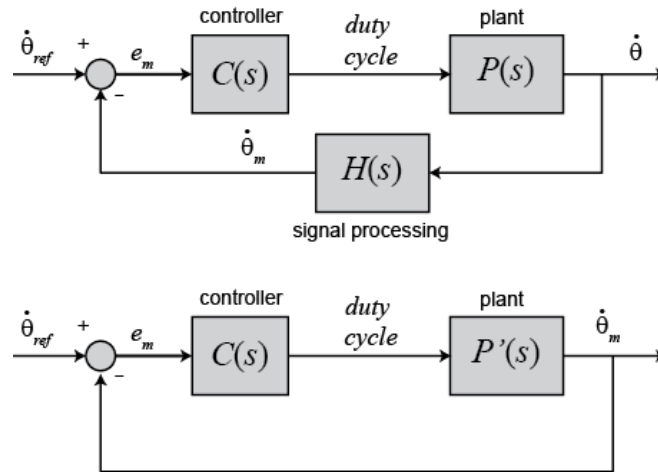
**Question 5** Does the estimated transfer function match the theory? Comment

### 3 Controller Design of a DC Motor

At a fundamental level, the voltage source (V) applied to the motor's armature is its input and the rotational speed of the shaft  $d\theta/dt$  is the output. Since in practice we are employing a Pulse-Width Modulation (PWM) approach to control, we will treat our control input as the PWM signal's duty cycle (percent of the PWM period for which the motor is "on"). The control input to the motor will be determined via a control law  $C(s)$  acting on the error between the commanded and measured motor speed. In this part, we investigated the processing needed for estimating the motor's speed. In this context, our closed-loop system would have the following form where the motor's speed is the true output, but significant processing (via  $H(s)$ ) is needed to generate the measured speed employed for feedback.

Since we have no means for measuring the motor's true speed, we will rather consider our closed-loop system to have the following format where our plant  $P'(s)$  includes the dynamics of the signal processing. We will determine a model for  $P'(s)$  in the same blackbox manner we did in the previous practical work. The specific transient controller requirements that we will design for are given below. If you are employing a physical motor significantly different than what we





will employ here, you may wish to alter the given requirements accordingly. We will also analyse the steady-state behaviour of the system, including in the presence of a constant disturbance.

- Settling time less than 0.5 second
- Maximum overshoot less than 20%

In this section we will design a PI controller, that is, we will choose the control gains  $K_i$  and  $K_p$  to place the closed-loop poles in some desired locations. since the PI controller makes our system Type 1, it will track a step reference with zero steady state error.

**Question 6** Convert the system's requirements into desired pole location

**Question 7** Use the pole placement method to find the value of  $K_p$  and  $K_i$

**Question 8** Using the Arduino IDE implement the PI controller, plot the response and the control effort, are all the requirements satisfied?, comment!

## 4 Robustness to disturbances

So far you have designed a PI/lag controller in the presence of a variety of real-world complications including uncertainties in the plant model and limits on the available control effort. Another factor that often must be considered in the implementation of control systems is the effect of unwanted inputs, such as disturbances and noise. In this section we will consider that our motor is subject to an unwanted load torque disturbance  $T_L$ .

**Question 9** Add the unwanted torque load  $T_L$  by applying constant pressure to the output motor shaft with our fingers. Plot the response and the control effort. Is your designed PI controller robust? comment!